

EXPENSE TRACKER

A Python + SQLite Database Project

Complete Project Report

Suitable for BCA / BCA AI-ML Students

Python • SQLite • SQL • CRUD

1. Abstract

Expense Tracker is a database-driven application developed using Python and SQLite. It allows users to record daily expenses, store them permanently, search transactions, delete records, and view spending summaries.

2. Introduction

Managing small daily expenses manually can be difficult. This project creates a simple digital system where every transaction is stored in an organized SQLite database.

3. Problem Statement

Manual expense records are difficult to maintain and summarize. Users need a quick way to record spending and understand where their money goes.

4. Objectives

- Add expenses with date, category, description and amount.
- Store records permanently in SQLite.
- View and delete transactions.
- Calculate total expenses.
- Generate category-wise and monthly summaries.
- Search saved expenses.
- Demonstrate Python and SQL skills.

5. Scope

The project is suitable for students and individuals who need a lightweight personal expense manager. It can later become a GUI, web or mobile application.

6. Technologies Used

Python 3: application logic and validation.

SQLite: local relational database.

SQL: data creation, insertion, retrieval, deletion and aggregation.

datetime: date validation and current-date handling.

7. System Requirements

Hardware: basic computer/laptop with 2 GB RAM or more. Software: Python 3.x and any code editor/terminal. No separate database server is required.

8. Functional Requirements

1. Add Expense
2. View All Expenses
3. Delete Expense

4. Show Total Expenses
5. Category Summary
6. Monthly Summary
7. Search Expenses
8. Exit

9. Database Design

The project uses one table named **expenses**. The ID uniquely identifies each transaction. Amount must be positive.

10. Table Structure

id — INTEGER PRIMARY KEY AUTOINCREMENT

date — TEXT in YYYY-MM-DD format

category — TEXT

description — TEXT

amount — REAL with a positive-value CHECK constraint

11. Program Flow

Start → Create database/table → Display menu → Select operation → Execute Python/SQL function → Display result → Return to menu → Exit.

12. Key Concepts

Functions, loops, conditions, exception handling, SQLite connectivity, parameterized SQL queries, CRUD, SUM, GROUP BY, LIKE, primary keys and database persistence.

13. Security/Good Practices

The program uses parameterized SQL placeholders (?) instead of directly joining user input into SQL statements. Input validation is applied to amounts and dates.

14. Complete Python Source Code

```
import sqlite3
from datetime import datetime

DB_NAME = "expenses.db"

def connect_db():
    return sqlite3.connect(DB_NAME)

def create_table():
    with connect_db() as conn:
        conn.execute("""
            CREATE TABLE IF NOT EXISTS expenses (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                date TEXT NOT NULL,
                category TEXT NOT NULL,
                description TEXT NOT NULL,
                amount REAL NOT NULL CHECK(amount > 0)
            )
        """)
        conn.commit()

def add_expense():
    category = input("Category: ").strip()
    description = input("Description: ").strip()
    amount_text = input("Amount (■): ").strip()

    try:
        amount = float(amount_text)
        if amount <= 0:
            raise ValueError
    except ValueError:
        print("Invalid amount. Please enter a positive number.")
        return

    date = input("Date (YYYY-MM-DD) [press Enter for today]: ").strip()
    if not date:
        date = datetime.now().strftime("%Y-%m-%d")

    try:
        datetime.strptime(date, "%Y-%m-%d")
    except ValueError:
        print("Invalid date format.")
        return

    with connect_db() as conn:
        conn.execute(
            "INSERT INTO expenses(date, category, description, amount) VALUES (?, ?, ?, ?)",
            (date, category, description, amount)
        )
        conn.commit()
    print("Expense added successfully.")

def view_expenses():
    with connect_db() as conn:
        rows = conn.execute(
            "SELECT id, date, category, description, amount "
            "FROM expenses ORDER BY date DESC, id DESC"
        ).fetchall()

    if not rows:
        print("No expenses found.")
        return

    print("\nID | Date          | Category      | Description                                     | Amount")
    print("-" * 75)
    for row in rows:
        print(f"{row[0]:<2} | {row[1]:<10} | {row[2]:<12} | "
              f"{row[3][:24]:<24} | ■{row[4]:.2f}")

def delete_expense():
    try:
        expense_id = int(input("Enter expense ID to delete: "))
    except ValueError:
        print("Invalid ID.")
        return

    with connect_db() as conn:
        cursor = conn.execute("DELETE FROM expenses WHERE id = ?", (expense_id,))
        conn.commit()
```

```

        if cursor.rowcount:
            print("Expense deleted successfully.")
        else:
            print("Expense ID not found.")

def total_expenses():
    with connect_db() as conn:
        total = conn.execute(
            "SELECT COALESCE(SUM(amount), 0) FROM expenses"
        ).fetchone()[0]
    print(f"Total expenses: ■{total:.2f}")

def category_summary():
    with connect_db() as conn:
        rows = conn.execute("""
            SELECT category, SUM(amount)
            FROM expenses
            GROUP BY category
            ORDER BY SUM(amount) DESC
            """).fetchall()

    if not rows:
        print("No expenses found.")
        return

    print("\nCategory Summary")
    print("-" * 35)
    for category, amount in rows:
        print(f"{category:<20} ■{amount:.2f}")

def monthly_summary():
    with connect_db() as conn:
        rows = conn.execute("""
            SELECT substr(date, 1, 7) AS month, SUM(amount)
            FROM expenses
            GROUP BY month
            ORDER BY month DESC
            """).fetchall()

    if not rows:
        print("No expenses found.")
        return

    print("\nMonthly Summary")
    print("-" * 35)
    for month, amount in rows:
        print(f"{month:<15} ■{amount:.2f}")

def search_expenses():
    keyword = input("Search category/description: ").strip()
    with connect_db() as conn:
        rows = conn.execute("""
            SELECT id, date, category, description, amount
            FROM expenses
            WHERE category LIKE ? OR description LIKE ?
            ORDER BY date DESC
            """, (f"%{keyword}%", f"%{keyword}%")).fetchall()

    if not rows:
        print("No matching expenses found.")
        return

    for row in rows:
        print(f"ID:{row[0]} | {row[1]} | {row[2]} | {row[3]} | ■{row[4]:.2f}")

def main():
    create_table()

    while True:
        print("""
===== EXPENSE TRACKER =====
1. Add Expense
2. View All Expenses
3. Delete Expense
4. Show Total Expenses
5. Category Summary
6. Monthly Summary
7. Search Expenses
8. Exit
=====
""")
        choice = input("Enter your choice: ").strip()

```

```
    if choice == "1":
        add_expense()
    elif choice == "2":
        view_expenses()
    elif choice == "3":
        delete_expense()
    elif choice == "4":
        total_expenses()
    elif choice == "5":
        category_summary()
    elif choice == "6":
        monthly_summary()
    elif choice == "7":
        search_expenses()
    elif choice == "8":
        print("Thank you for using Expense Tracker!")
        break
    else:
        print("Invalid choice. Please try again.")

if __name__ == "__main__":
    main()
```

15. Sample Output

```
===== EXPENSE TRACKER =====
1. Add Expense
2. View All Expenses
3. Delete Expense
4. Show Total Expenses
5. Category Summary
6. Monthly Summary
7. Search Expenses
8. Exit
=====
Enter your choice: 1
Category: Food
Description: Lunch
Amount (■): 120
Date (YYYY-MM-DD) [press Enter for today]:
Expense added successfully.

Enter your choice: 4
Total expenses: ■120.00
```

16. Testing

- Valid expense is inserted successfully.
- Zero/negative amounts are rejected.
- Invalid dates are rejected.
- Saved records appear in View.
- Existing IDs can be deleted.
- Unknown IDs produce an error message.
- SUM gives the total spending.
- GROUP BY produces category/month summaries.
- Search returns matching category or description.

17. Advantages

Simple, lightweight, offline, persistent, beginner-friendly, and easy to extend. It demonstrates practical Python + SQL skills.

18. Limitations

Command-line interface only; single local database; no authentication; no budget alerts; no graphical charts in the current version.

19. Future Enhancements

Tkinter GUI, income tracking, budgets and alerts, Matplotlib charts, CSV/PDF export, recurring expenses, login/authentication, cloud backup, and web/mobile deployment.

20. Conclusion

The Expense Tracker combines Python programming and SQLite database management into a useful real-world application. It is a strong beginner-to-intermediate BCA portfolio project and provides a foundation for advanced financial-management features.

21. Viva Questions

1. What is SQLite? — A lightweight relational database engine.
2. What is CRUD? — Create, Read, Update and Delete.
3. Why use sqlite3? — It is available in Python's standard library.
4. What is a primary key? — A unique identifier for a row.
5. What does SUM do? — Adds numeric values.
6. Why GROUP BY? — To create grouped summaries.
7. Why use ? placeholders? — For parameterized and safer SQL queries.
8. What does AUTOINCREMENT do? — Generates IDs automatically.
9. Why COALESCE? — Converts a NULL total into 0.
10. How can it be upgraded? — Add GUI, charts, budgets, authentication and exports.

22. Project Folder

```
Expense_Tracker_Project/  
■■■ main.py  
■■■ README.md  
■■■ Expense_Tracker_Project_Report.pdf  
■■■ expenses.db    (created automatically when main.py runs)
```